

Project Report

Flight Delay Prediction

Group 7 - Data Nuggets
Cora Wackermann
Kejsi Limaci
Simon Schumacher
Justus Mrosk
Nick Birkholz
Juan Cordero

May 17th, 2026

Submitted to
Data and Web Science Group
Prof. Christian Bizer
University of Mannheim

Contents

1	Application Area and Goals	2
2	Profile of Dataset	2
2.1	Filtering Strategies	2
2.2	Structure of the Dataset	3
3	General Preprocessing Steps	3
3.1	Data Cleaning	4
3.2	Feature Engineering	4
3.3	Target Variable Selection and Distribution Analysis	5
3.4	Correlation Analysis	5
3.5	Data Transformation	5
3.6	Train-Test Split	5
4	Baseline Model	6
5	Machine Learning Models	6
5.1	Chosen Models	6
5.2	Hyperparameter Tuning and Cross Validation	7
6	Evaluation Setup and Results	7
6.1	Results	8
6.2	Feature Importance	9
6.3	Error Analysis	10
6.4	Comparison with State of the Art	10
7	Conclusion	11
	References	12

1 Application Area and Goals

Optimizing flight schedules is a fundamental challenge for the aviation industry, requiring an accurate understanding of potential delays. These disruptions, typically arising from a complex interplay of weather conditions and operational inefficiencies, lead to significant financial losses and reputational damage for airlines or airport operators. To address this, our project aims to forecast the arrival delay severity of a flight two hours before scheduled departure. Accurate forecasts help stakeholders optimize ground operations and reduce these losses. Furthermore, such a model empowers consumers by providing greater transparency regarding flight punctuality. We frame this as a multi-class classification task with four severity levels.

This paper provides insights into predictive modeling approaches for this task. The model uses real-time weather observations available no later than the prediction cutoff time, together with historical flight patterns, to classify delays into four categories of severity. The goals of this paper are to: (1) select and profile a suitable large-scale flight dataset; (2) define the target variable and apply preprocessing steps, including data cleaning and feature engineering; (3) develop two rule-based baseline systems and train several machine learning models; (4) perform hyperparameter optimization; and (5) validate model performance, primarily using the F2-score, followed by a critical evaluation of the results.

2 Profile of Dataset

The primary data source is the “Carrier On-Time Performance” dataset from the American Bureau of Transportation Statistics ([Bureau of Transportation Statistics 2026](#)). It contains comprehensive information on flight logistics, including the airline carrier, origin and destination airports, aircraft tail number and flight distance. Most importantly, it tracks on-time performance metrics such as planned versus actual departure and arrival times.

For data acquisition, we downloaded the raw CSV files for all months from 2013 to 2019 from the dataset homepage. These monthly files were then combined into a single dataframe to create one consistent raw data foundation for further preprocessing and feature engineering. To support efficient storage and repeated access in the project pipeline, the merged dataset was subsequently saved as a Parquet file in S3 Object Storage.

The raw dataset consists of 37,559,401 rows and 109 columns, to which we apply a filtering mask implementing the strategies described in the following section.

2.1 Filtering Strategies

To maintain a feasible scope and ensure high data quality, we implemented the following filtering strategies:

- **Temporal Range:** We limit the data to the years 2014–2019, while using some

data from 2013 for feature engineering, such as rolling averages. This time frame avoids extreme anomalies in flight patterns caused by the COVID-19 pandemic.

- **Geographic and Operational Scope:** The data is filtered to focus on 35 major U.S. airports and 7 major national airlines (American, Southwest, JetBlue, Alaska, United, Delta and Frontier). This reduces noise from infrequent regional flights and creates a more consistent dataset for training and testing.
- **Anomaly Removal:** To ensure data completeness and improve training stability, we removed all canceled and diverted flights. We applied this filtering to both training and test data. This is a deliberate simplification of real-world operations that should be noted as a limitation.
- **Removal of columns:** Many columns in the raw data do not provide useful information for this use case and were therefore removed. For instance, we dropped all diversion-related columns because diverted flights are not included in our training data.

2.2 Structure of the Dataset

The raw dataset contains operational flight information, scheduled and realized time variables, delay measurements, distance information, airline and airport identifiers, timezone information, and weather-related timestamps.

Feature Name	Type	Explanation
FlightDate	str	Local day of flight
Reporting_Airline	str	Airline operating the flight
Tail.Number	str	Aircraft identifier
Flight_Number_Reporting_Airline	object	Flight number, without airline id
Origin, Dest	str	Origin and Destination airports
CRSDepTime	object	Local time of scheduled departure
DepTime	object	Local time when the plane actually departed
DepDelayMinutes	int	Minutes the plane departed late
CRSArrTime	object	Local time of scheduled arrival
ArrTime	object	Local time of actual arrival
ArrDelayMinutes	int	Minutes the plane arrived late
Distance	float	Distance of the flight
Delay Reasons (Lateaircraft, Carrier, NAS, Weather, Security)	object	Five features indicating the minutes of delay attributed to each reason

Table 1: Overview of selected features, data types and descriptions.

3 General Preprocessing Steps

The dataset described above was preprocessed to prepare it for training flight-delay prediction models. This section outlines the preprocessing steps applied.

3.1 Data Cleaning

After applying the filtering rules, the remaining records were cleaned to correct technical inconsistencies. **Missing values** were present in some records: rows lacking essential scheduling information were removed, while missing values in derived historical or airport-level features were handled later during feature engineering through imputation. Furthermore, **invalid values** were found in time-related features and converted accordingly. **Target leakage** was prevented by removing all post-departure and post-arrival variables (realized times, taxi durations, wheels-off/on times, delay reason columns), since these would not be available at prediction time. **Duplicate checks** found no duplicate records. **Data normalization** converted time columns into consistent datetime formats aligned across local and UTC time zones to enforce the two-hour prediction cut-off reliably. **Outlier removal** was not needed because the target variable was binned into discrete classes.

3.2 Feature Engineering

Six feature categories were engineered to capture different causes of delay, yielding a total of 86 features.

Real-time airport delay statistics. Congestion at an airport spreads to all flights departing later that day. Four cumulative delay measures are tracked in hourly snapshots. Each flight receives the snapshot from the last completed hour before the two-hour cutoff, so its own outcome never leaks into the features.

Aircraft chain effects. The same aircraft operates several flights in sequence, so a delay on one leg carries over to the next. Tail-number matching provides the previous flight’s departure status, delay, and expected turnaround time.

Calendar and holiday indicators. Passenger volume typically increases around holidays while staffing may decrease. A binary holiday flag and the number of days to the next holiday are included.

Weather conditions. Hourly observations (temperature, wind speed, precipitation) for origin and destination airports are retrieved from the Meteostat library ([Meteostat Developers 2026](#)), sourced from NOAA ([National Oceanic and Atmospheric Administration 2026](#)), at two hours before scheduled departure. Missing values are addressed through nearby-station fallback and forward-filling up to two hours.

Rolling historical delay averages. Certain airlines, origins, and destinations tend to experience delays consistently over time. An expanding mean, a 7-day rolling mean, and a 30-day rolling mean of past delays are computed at each level with a one-day shift to prevent same-day leakage.

Lag features. Delay patterns tend to repeat across consecutive days and on the same weekday the following week. Lag-1 and lag-7 delay values are computed per origin, destination, carrier, and globally.

3.3 Target Variable Selection and Distribution Analysis

We use *ArrDelayMinutes* as the target variable, since arrival delay directly affects gate assignments, connections, and downstream scheduling. The continuous variable is discretized into four ordered bins (Table 2). The resulting distribution is strongly right-skewed, with the majority class accounting for over 80% of flights:

Delay class	Arrival delay	Distribution
No delay	≤ 15 minutes	80.8%
Light delay	> 15 and ≤ 60 minutes	14.1%
Medium delay	> 60 and ≤ 180 minutes	3.8%
High delay	> 180 minutes	1.3%

Table 2: Delay classes used for the target variable *delay_class*.

The dataset also contains information about the reasons for the delays. According to that, about 40% of delays, are due to late arriving aircraft, about 30% due to factors related to the airline, about 20% related to air traffic control and 10% related to weather (Thiagarajan et al. 2017).

3.4 Correlation Analysis

Pearson correlations between numerical features and *ArrDelayMinutes* show that aircraft-rotation variables (previous-flight departure status, delay, turnaround time) and recent airport-delay averages exhibit the strongest linear association with the target. This supports the assumption that delay propagation and short-term congestion are key drivers. Highly correlated inter-feature pairs were removed to reduce redundancy. Further feature subsampling is handled internally by XGBoost and Random Forest.

3.5 Data Transformation

Before using the data for training, some features still need to be transformed. Categorical variables, such as airlines and airports, are one-hot encoded, while timestamps and continuous variables, are turned into continuous Gaussian distributed variables, using a Standard Scaler. Some boolean or discretely valued features are left, to preserve specific meanings tied to their values, such as months. The transformations are fitted on the training data to prevent leakage, and then only applied to the testing data.

3.6 Train-Test Split

The final step of the preprocessing phase was to split the dataset into temporally ordered training and test sets. Since flight delay prediction is inherently a forecasting task, the split was performed chronologically rather than through stratified or random sampling. Flights from 2014 through 2018 were used for training and validation, while flights from 2019 were reserved exclusively for final testing. This time-based split reflects the real-world prediction setting more faithfully, because it ensures that the model is always

evaluated on future observations rather than on randomly mixed records from the same period. This procedure prevents temporal leakage and yields a more realistic estimate of model performance under operational conditions.

4 Baseline Model

Before training the main classification models, we defined two baseline approaches to establish transparent reference points for evaluating predictive performance.

The first baseline is a dummy classifier that always predicts the most frequent class. Due to the strong class imbalance in the dataset, where the majority of flights have an arrival delay of less than 15 minutes, this simple strategy will therefore, correctly classify most datapoints, however it will not have any real use for prediction.

The second baseline uses recent airport-level delay information. For each target flight, the model considers the average departure delay of previously observed outgoing flights from the same origin airport up to the prediction time. This is defined as the start of the day until two hours before scheduled departure. This average delay is then mapped to the corresponding delay class and used as the prediction for the target flight. The underlying assumption is that flights departing from the same airport within a similar temporal context may be affected by comparable operational conditions.

These approaches were chosen for their simplicity and challenge to the imbalance of the underlying data, while being naive, problem-oriented strategies.

5 Machine Learning Models

In the next step, several machine learning models were selected and trained to predict flight delay severity. The general training process was the following. First, a diverse group of algorithms was selected to compare different levels of complexity and interpretability. After this, hyperparameter optimization was applied using time-series-aware validation folds to prevent temporal data leakage, while class-imbalance handling in the form of class weights, was integrated into the modeling pipeline.

5.1 Chosen Models

Since the target variable is a discrete categorical variable, several classification approaches were trained to predict flight delay severity. For this purpose, the following machine learning methods were selected:

- **Linear and Probabilistic Models:** Logistic Regression to evaluate linear relationships within high-dimensional encoded categorical data, and Naïve Bayes to analyze class probabilities based on marginal feature distributions.
- **Tree-Based Models for Classification:** A single Decision Tree to derive explicit, symbolic decision rules, and Random Forest as a robust bagging ensemble to capture non-linear interactions while reducing variance.

- **Tree-Based Gradient Boosting Architectures:** Histogram-based Gradient Boosting for scalable and computationally efficient feature binning, and XGBoost to optimize predictive performance across highly heterogeneous feature groups.

5.2 Hyperparameter Tuning and Cross Validation

Model	Search space
Logistic Regression	$C \sim \log \mathcal{U}(10^{-3}, 10)$
Decision Tree	criterion { gini , entropy , log_loss }; depth {None, 8, 12, 16, 24}; min. split {2, 10, 25, 50}; min. leaf {1, 5, 10, 25}; max. features {None, sqrt , log2 }; ccp alpha {0, 0.001, 0.01}
Naïve Bayes	variance smoothing $\sim \log \mathcal{U}(10^{-12}, 10^{-6})$
Random Forest	$n_{\text{estimators}} \in \{100, 200, 400\}$; depth $\in \{\text{None}, 12, 24\}$; min. leaf $\in \{5, 10, 25\}$; max. features $\in \{\sqrt{\cdot}, \log_2\}$
Hist. Gradient Boosting	learning rate $\in \{0.03, 0.06, 0.10\}$; iterations $\in \{100, 200, 400\}$; leaf nodes $\in \{15, 31, 63\}$; min. leaf $\in \{20, 50, 100\}$; $L_2 \in \{0, 0.01, 0.1\}$
XGBoost	learning rate $\in [0.02, 0.10]$; estimators $\in \{300, 400, 600, 800\}$; depth $\in [4, 8]$; min. child weight $\in \{1, 3, 5, 10\}$; subsample / colsample $\in \{0.70, 0.85, 1.00\}$; $\lambda \in \{1, 3, 10, 30\}$; $\alpha \in \{0, 0.1, 1\}$; $\gamma \in \{0, 0.5, 1, 2\}$

Table 3: Compact hyperparameter search spaces used during randomized tuning.

All models went through a structured hyperparameter tuning process to find the best configuration for each model. Since the dataset is chronologically ordered, random cross-validation was not used as it would allow future data to influence model selection, a problem known as temporal data leakage. Instead, time-series-aware validation folds were applied, where each validation period always comes after its training period.

Randomized search was used to explore the hyperparameter search space shown in Table 3 without testing every possible combination. Each model was tuned using 20 randomly sampled configurations and three-fold time-series cross-validation. The search space was tailored to each model, including regularization strength for Logistic Regression, tree depth and feature sampling for Random Forest, and learning rate for boosting models. Class imbalance was handled inside the pipeline so that minority delay classes received appropriate weights.

To allow a fair comparison across models, all tuning was carried out using the same data split, feature set, preprocessing steps, evaluation metric, and random seed.

6 Evaluation Setup and Results

Macro-based scoring gives each delay class comparable importance and therefore does not allow the majority class to dominate model selection. We chose macro-F2 as our primary metric because it gives greater weight to recall and therefore rewards a more risk-aware prediction strategy, since it punishes false negatives more than false positives. In this context, missing an actual delay is more costly than issuing a false delay warning, which is desirable in this case. To provide a broader picture of model performance, we also report the accuracy, and class specific precision and recall scores of the models.

As mentioned before, our test set is composed of the held-out year of 2019, to have a consistent data from a whole year, to avoid seasonal effects.

6.1 Results

Model	F2	Acc	Precision				Recall			
			C0	C1	C2	C3	C0	C1	C2	C3
XGBoost	0.515	0.789	0.903	0.364	0.480	0.416	0.884	0.420	0.452	0.411
XGBoost (no weights)	0.442	0.838	0.861	0.583	0.693	0.707	0.987	0.193	0.327	0.243
Hist. Gradient Boosting	0.505	0.777	0.910	0.358	0.422	0.371	0.867	0.431	0.514	0.435
Random Forest	0.488	0.733	0.904	0.306	0.516	0.305	0.833	0.461	0.449	0.435
Decision Tree	0.402	0.606	0.904	0.214	0.275	0.067	0.657	0.391	0.380	0.551
Naïve Bayes	0.353	0.687	0.855	0.208	0.165	0.076	0.809	0.151	0.234	0.412
Logistic Regression	0.272	0.415	0.932	0.161	0.089	0.221	0.444	0.180	0.834	0.102
Baseline: Majority class	0.238	0.804	0.803	0.0	0.0	0.0	1.0	0.0	0.0	0.0
Baseline: Avg of day so far	0.258	0.786	0.814	0.221	0.263	0.0	0.964	0.081	0.006	0.0

Table 4: Model performance on the test set (macro-F2, accuracy, and per-class precision/recall). Classes: C0 = No delay, C1 = Light delay, C2 = Medium delay, C3 = High delay.

Table 4 summarizes the macro-F2 score and accuracy of all evaluated models on the held-out test set. XGBoost with class weighting achieves the highest macro-F2 of 0.5150 at an accuracy of 0.7893. The variant without class weights attains a higher accuracy of 0.8376, yet its macro-F2 drops substantially to 0.4417, illustrating that accuracy is a misleading indicator under strong class imbalance. Compared to both baselines, class-weighted XGBoost yields a considerably higher macro-F2, demonstrating that it identifies minority delay classes more reliably, even though its overall accuracy falls marginally below that of the majority-class dummy classifier.

Histogram-based Gradient Boosting performs similarly, achieving a macro-F2 of 0.5050, placing it in the same performance tier as XGBoost. More broadly, ensemble methods consistently outperform both linear and shallow models across both evaluation metrics.

The decision tree achieves a macro-F2 of 0.4022 and an accuracy of only 0.6059. Inspection of the learned tree structure reveals that expected turnaround time serves as the dominant split criterion, suggesting that the model’s predictive logic relies almost entirely on aircraft rotation dynamics rather than capturing the broader set of delay patterns present in the data.

Across all models, precision for the majority class (C0) remains consistently high, while minority classes show substantially lower and more variable scores. Since the F2 score weights recall twice as heavily as precision, the ranking of models reflects that class-weighted XGBoost achieves the highest F2 by maintaining balanced recall across all four classes rather than maximising C0 precision. A notable anomaly appears in Logistic Regression, which achieves the highest C2 recall (0.834) at near-zero precision (0.089), yet its macro-F2 remains the lowest among trained models, as strong recall on a single minority class does not compensate for poor performance elsewhere. Similarly,

XGBoost without class weights attains near-perfect C0 recall (0.987) by collapsing most predictions into the majority class, suppressing minority recall and thus its F2.

6.2 Feature Importance

Since XGBoost achieved the best performance in terms of macro-F2 score, the feature importance analysis was mainly conducted for this model. The results show that the most influential predictors are related to the previous flight of the same aircraft, especially whether the previous flight had already departed, the previous delay, and the expected turnaround time as shown in Table 1.

This pattern is also visible in the other tree-based models. Random Forest and the Decision Tree both assign high importance to turnaround-related and previous-flight features. This suggests that delay propagation within the aircraft rotation is one of the strongest signals in the dataset.

Besides previous-flight information, historical delay statistics also appear among the relevant predictors. These features capture recurring delay patterns at the flight, route, or airport level. Weather variables and airport indicators are present as well, but they are less dominant than operational and historical delay features.

Overall, the feature importance results indicate that the model mainly relies on aircraft rotation information and recent delay history. This is plausible for flight-delay prediction, because delays often carry over from one flight to the next when the available turnaround time is limited.

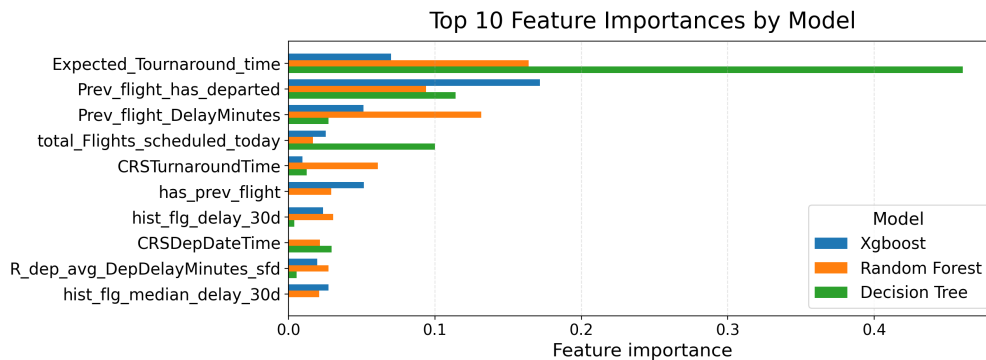


Figure 1: Top 10 feature importances of the best-performing XGBoost model.

6.3 Error Analysis

Actual / Predicted	No delay	Light delay	Medium delay	Large delay
No delay	1,471,144	145,917	33,021	7,937
Light delay	135,256	93,622	32,194	2,443
Medium delay	31,341	24,923	52,834	4,715
Large delay	6,104	2,495	7,865	9,202

Table 5: Confusion matrix for the four delay-severity classes. Rows represent actual classes and columns represent predicted classes.

While the models improve on baseline predictions, the results remain limited. The most noticeable challenge is distinguishing between no-delay and light-delay classes. This is expected, as the 15-minute threshold lies within normal operational variation, making it difficult to determine whether minor disruptions will cross this boundary, as displayed in the confusion matrix in Table 5. The medium delays of one to three hours are better identified by the model, while for the large delay prediction, performance declines again. This might be since flights over three-hour delay are rather rare, and probably often very anomalous. The model therefore has to predict the effect of outliers.

6.4 Comparison with State of the Art

Recent studies on flight delay prediction often report substantially higher accuracy than the models in this study. For example, (Thiagarajan et al. 2017) uses the the same dataset, as in our study together with weather data and reports accuracies of up to 0.94 for predicting whether a flight is delayed or not. However, these results are not directly comparable to the present study. Their task is formulated as a binary classification problem, while our approach distinguishes between four delay-severity classes. This makes the prediction task considerably harder, especially because medium and large delays are rare in the dataset. Additionally, the study uses random or insufficiently described train-test splits, which may overestimate performance in a forecasting setting. In contrast, this study evaluates the models on a chronologically held-out test year, which better reflects real-world deployment.

A further relevant comparison is (Alonso and Loureiro 2015), which predicts departure delays at Porto airport using similar delay categories as ours. Although the reported performance is limited, the study identifies similar feature groups as important, especially previous delay information and operational flight characteristics. This is consistent with our feature importance analysis, where aircraft-rotation features and recent delay history dominate the prediction.

Overall, the results of this study are lower than some reported state-of-the-art accuracies, but they are obtained under a stricter and more realistic evaluation setup. The best model, class-weighted XGBoost, reaches a macro-F2 score of 0.515 and an accuracy of 0.789. While this is not sufficient for reliable operational deployment, it shows that delay-severity prediction two hours before departure is possible and that

tree-based ensemble models can extract useful signals from historical, operational, and weather-related features.

7 Conclusion

This study investigated whether flight delay severity can be predicted two hours before scheduled departure using weather observations, historical delay patterns, airport-level congestion features, and aircraft-rotation information. The results show that the proposed machine learning models improve over both rule-based baselines, especially when evaluated with macro-F2. The best-performing model, XGBoost with class weighting, achieved the highest macro-F2 score, indicating that the engineered features contain useful predictive information beyond the majority-class baseline.

However, the overall predictive performance remains limited. The feature importance analysis shows that the models rely strongly on aircraft-rotation variables and recent delay history. This suggests that the approach captures delay propagation reasonably well, but it does not fully explain other major sources of delay. Important factors such as crew availability, maintenance issues, carrier-specific operational disruptions, air traffic control restrictions, runway closures, and broader airport capacity constraints are only partially represented or missing from the dataset.

A further limitation is that the dataset only contains domestic scheduled flights. International, charter, military, and other non-included flights may still affect airport congestion and aircraft availability, but they are not visible in the data. As a result, traffic conditions and turnaround processes may be estimated incompletely. In addition, canceled and diverted flights were removed, which simplifies the prediction task but limits the models applicability to real-world disruption management.

Overall, the thesis goal was partially achieved: the models outperform simple baselines and demonstrate that delay severity prediction is possible under a two-hour forecasting constraint. Nevertheless, the results are not yet sufficient for reliable operational deployment. Future work should incorporate richer operational data, especially information on carrier operations, air traffic control constraints, runway availability, and complete aircraft rotations, to improve the practical usefulness of the prediction system.

References

- Alonso, H. and A. Loureiro (2015). Predicting flight departure delay at porto airport: A preliminary study. In *Proceedings of the 7th International Joint Conference on Computational Intelligence*, pp. 93–98. SciTePress.
- Bureau of Transportation Statistics (2026). Carrier On-Time Performance. Online database. Accessed 16 May 2026. Available at: https://www.transtats.bts.gov/Tables.asp?Q0_VQ=EFD.
- Meteostat Developers (2026). Meteostat Developers Documentation. Online documentation. Accessed 16 May 2026. Available at: <https://dev.meteostat.net/>.
- National Oceanic and Atmospheric Administration (2026). National Oceanic and Atmospheric Administration. Website. Accessed 16 May 2026. Available at: <https://www.noaa.gov/>.
- Thiagarajan, B., L. Srinivasan, A. V. Sharma, D. Sreekanthan, and V. Vijayaraghavan (2017). A machine learning approach for prediction of on-time performance of flights. In *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
ChatGPT	Rephrasing and grammar	Throughout	+
DeepL	Translation	Throughout	+
Claude	Code refactoring	Python code	++